

CSI243 Functional Programming

Lab I Introduction to the Haskell Environment

Objectives

1. **Start Haskell**
2. **Basic expressions**

Background / Scenario

As this is the very first lab, we will be looking at evaluating basic expressions and functions using Haskell.

Task1: Start GHCi

To start GHCi, type `ghci` at the prompt. You can also start it by choosing it from the Start menu.

What is the prompt displayed and why?

Task 2: GHCi commands

GHCi commands begin with a colon `:`. Most of the commands can be abbreviated to only the initial letter. The command `:help`, abbreviated to `:h` or `:?` is used to display the complete list of the GHCi commands. Use this to find what the following commands mean and if they can also be abbreviated:

a. `:load`

b. `:type`

c. `:quit`

d. `:info`

Task 3: Evaluate expressions

As a functional programming language, Haskell works like a functional calculator.

1. What do the following expressions evaluate to?

- i. `1+2*3+4` _____
- ii. `400>200` _____
- iii. `1/=1` _____
- iv. `True || False` _____

2. These following expressions are not valid Haskell expressions. Why? What is the correct expressions?

- i. `9 + -7`

- ii. `X == x`

- iii. `False || true`

- iv. `If 'A' > 'a' then True`

3. Create an expressions that adds 5 to the sum of 19 and 1 and multiplies the results by 10. What does it evaluate to?

Task 4: Create basic functions

1. Create a function called `cube` which takes a number `x` and gives its cube:

```
Prelude> cube x = x*x*x
```

- i. Run the function using different values of `x`.
- ii. Use `:type cube` to find the type of the function

2. Write a function `isNegative` that determines if a number is negative, that is, if the number is less than 0.

3. Write a function `power x n` that raises `x` to the power of `n`. What is its type?

Task 5: Hello World

At the prompt, do the following and write down what they do:

- i. Prelude> `Hello World`

- ii. Prelude> `"Hello World"`

- iii. Prelude> `print Hello World`

- iv. Prelude> `print "Hello World"`

- v. Prelude> `Print "Hello World"`

- vi. Prelude> `putStrLn "Hello World"`
